DESIGN OF DIGITAL SYSTEMS

ΠΑΝΕΠΙΣΤΗΜΙΟ ΔΥΤΙΚΗΣ ΑΤΤΙΚΗΣ
UNIVERSITY OF WEST ATTICA

DEPARTMENT OF INFORMATION AND COMPUTER ENGINEERING

TASK 4
DESIGN OF SEQUENTIAL CIRCUITS AND REGISTERS FILES

STUDENT DETAILS

NAME: ATHANASIOU VASILEIOS EVANGELOS
STUDENT ID: 19390005
STUDENT **SEMESTER:** 8th
STUDENT STATUS : UNDERGRADUATE
STUDY PROGRAM : UNIWA
LABORATORY DEPARTMENT : DEPARTMENT 1 WEDNESDAY 17:00 – 19:00
LABORATORY MANAGER : MILIDONIS ATHANASIOS
DELIVERY DATE : 30/5/2023

STUDENT PHOTO:

1. Design a simple 4-bit register. The register will consist of D-type flip flops and will have the following interface and structure

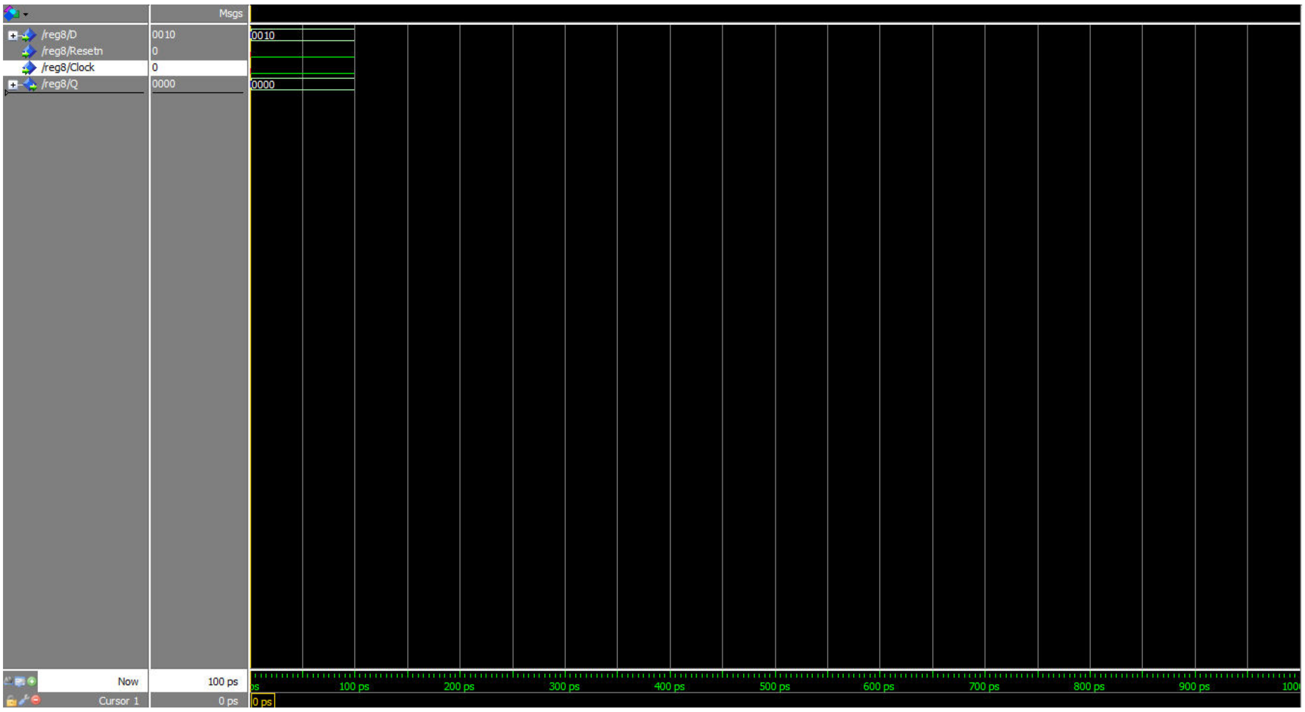
```
LIBRARY ieee ;
USE ieee.std_logic_1164.all;

ENTITY reg8 IS
PORT (
D                : IN std_logic_ vector ( 3 DOWNTO 0);
                Resetn , Clock   : IN std_logic ;
Q                : OUT std_logic_ vector ( 3 DOWNTO 0));
END reg8;

ARCHITECTURE behavioral OF reg8 IS
BEGIN
PROCESS ( Resetn , Clock )
BEGIN
IF Resetn = '0' THEN
Q <= "0000"?
ELSIF rising_ edge ( Clock) THEN
Q <= D;
END IF?
END PROCESS?
  END behavioral?
```

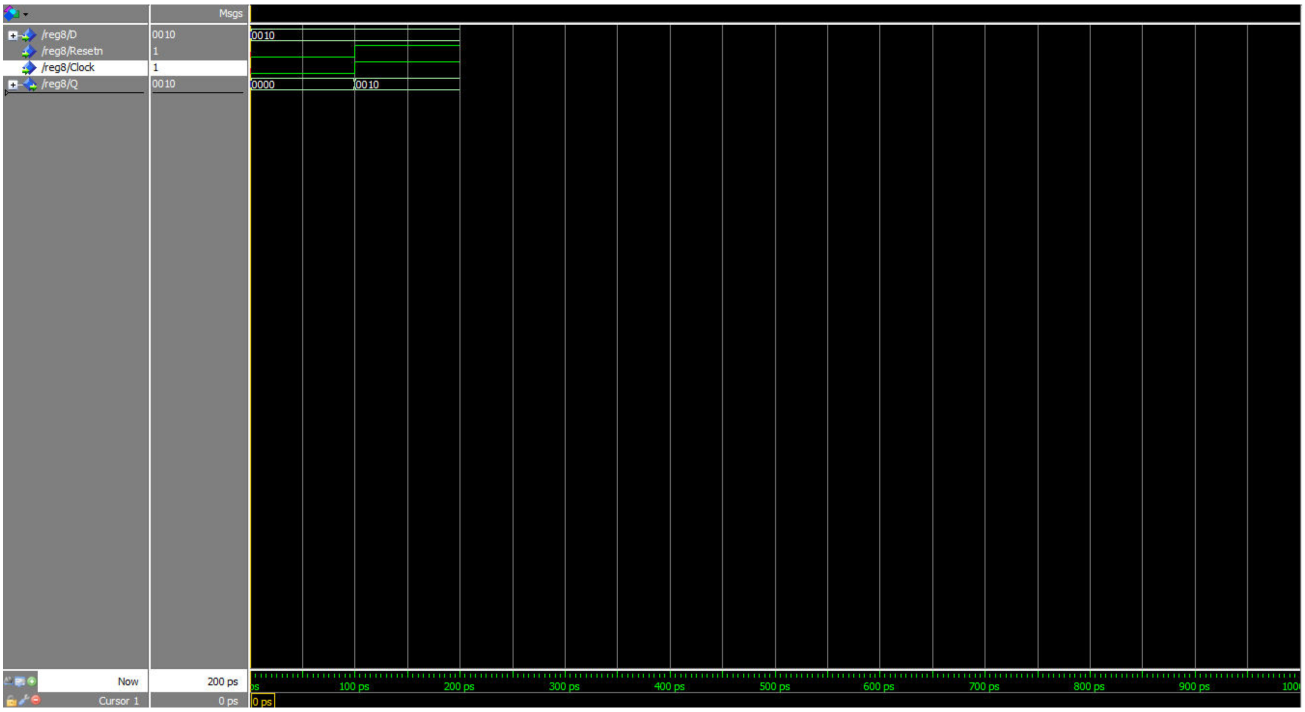
2. Test the above unit by putting the following as inputs and observing the outputs.

In	Out
0010	0010
1110	1110
1010	1010

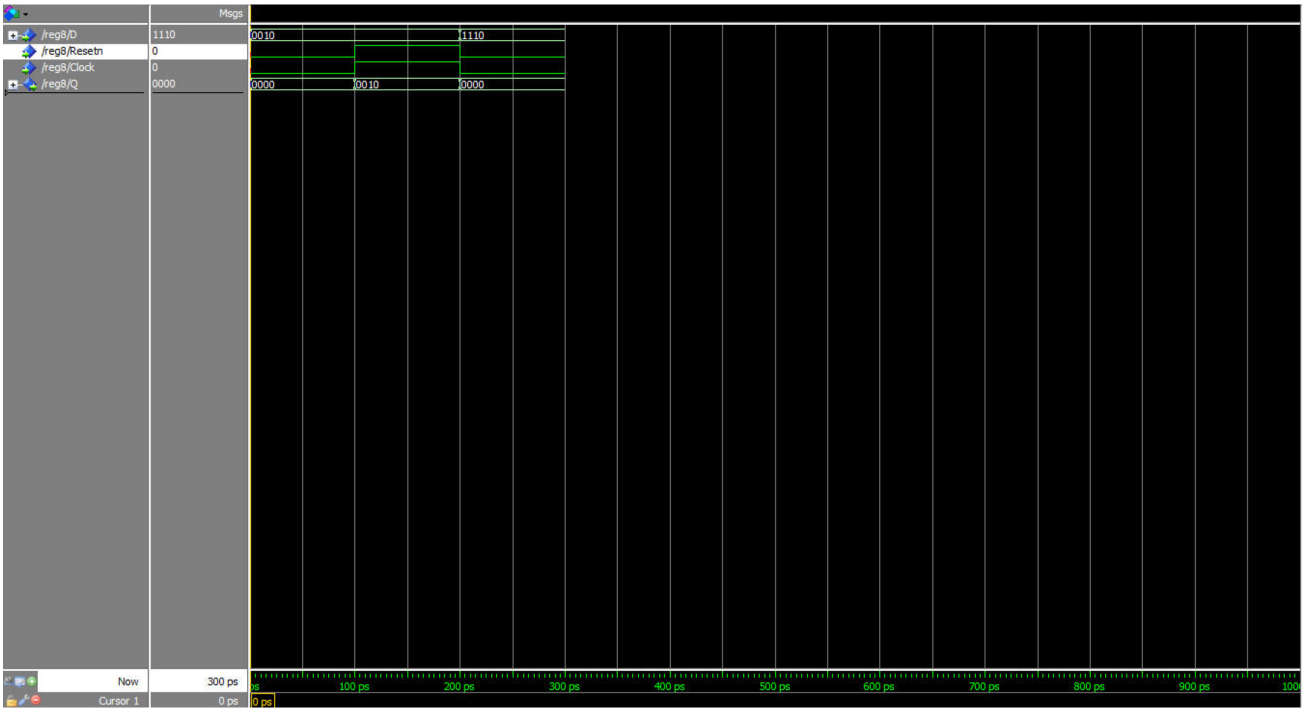
The input transitions to the output with the clock rising and the " Resetn " bit set to "1".

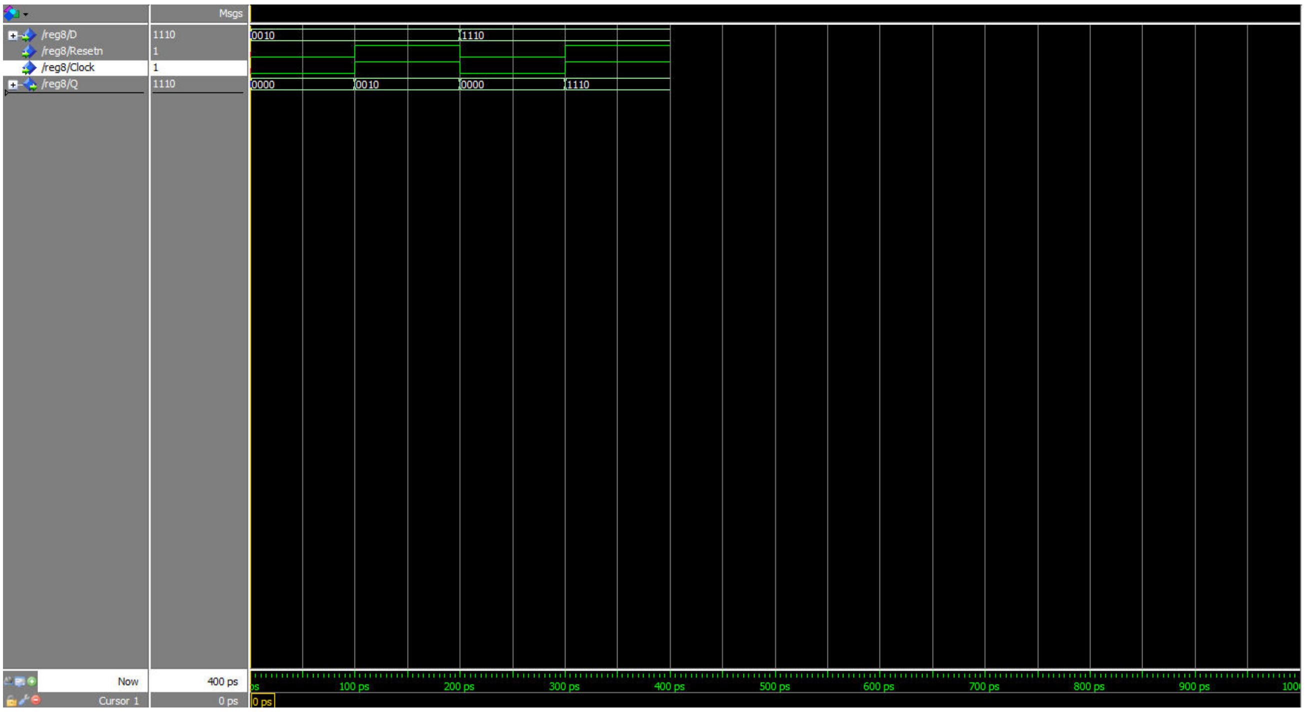
Clock / In: 0010



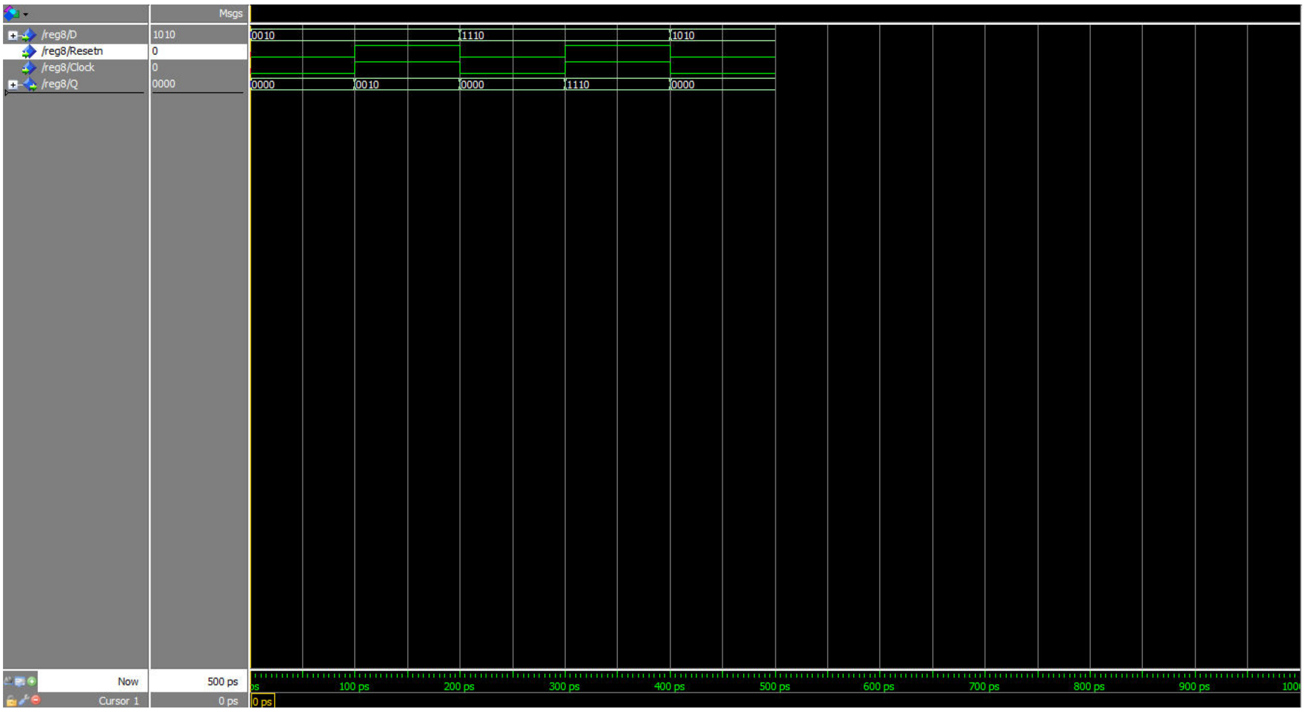


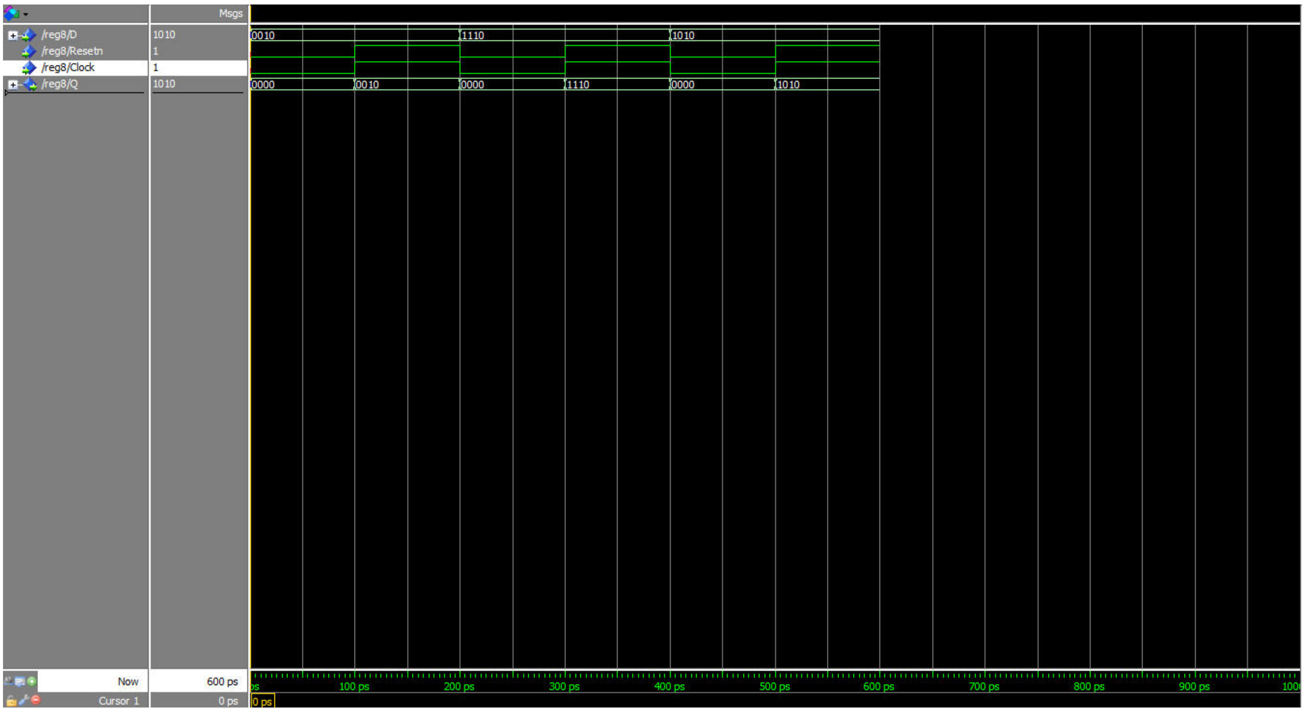
Clock / In: 11 10





Clock / In: 1 010



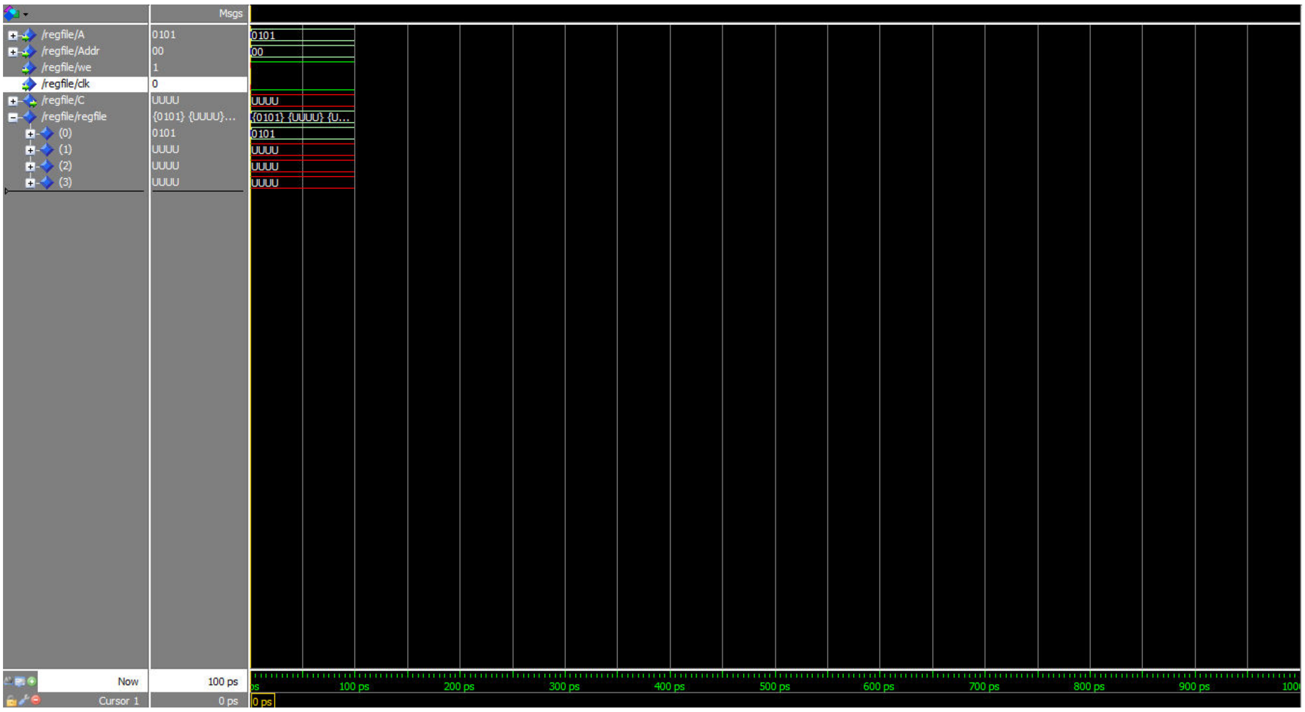


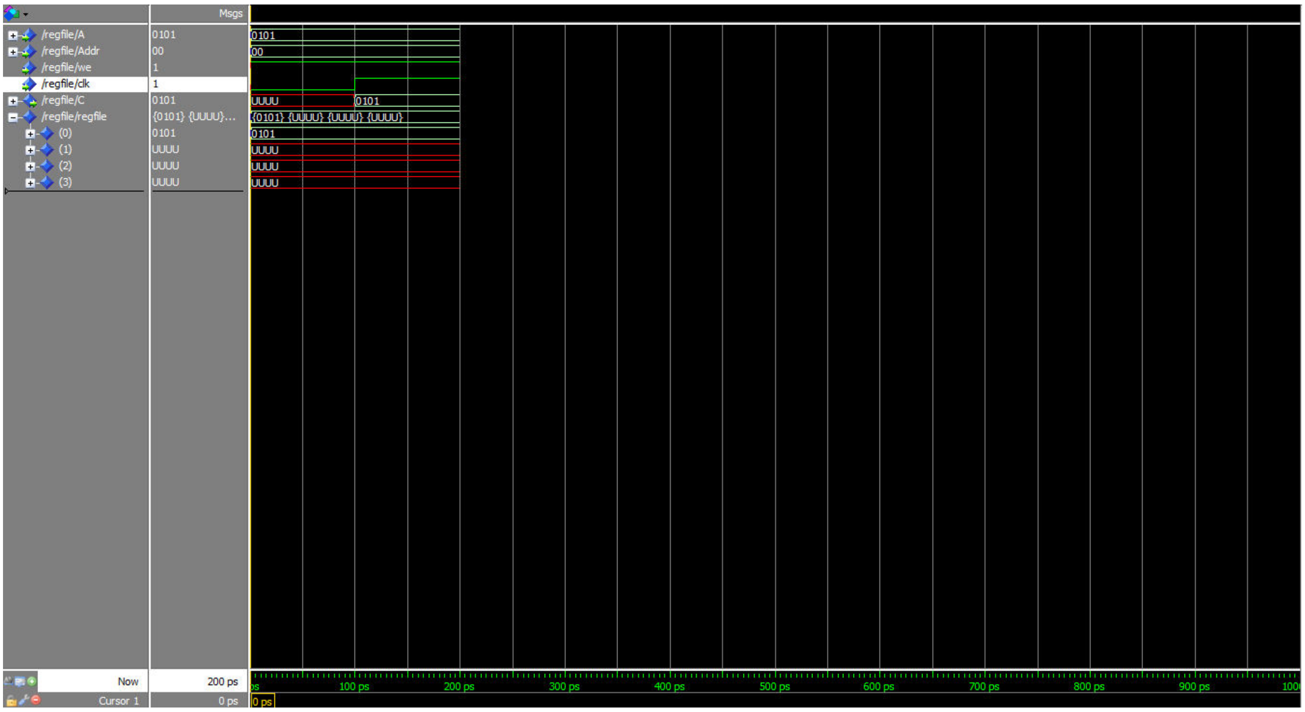
3 . The following VHDL code creates a register file of 4 registers, each consisting of 4 bits.

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all;
USE ieee.std_logic_unsigned.all ;
USE ieee.numeric_std.all ;

ENTITY regfile IS
GENERIC ( dw : natural := 4;
            size : natural := 4;
            addrw : natural := 2);
    PORT ( A : IN std_logic_vector (dw-1 DOWNTO 0);
            Addr : IN std_logic_vector (adrw-1 DOWNTO 0);
we : IN std_logic ;
            clk : IN std_logic ;
C : OUT std_logic_vector (dw-1 DOWNTO 0));
END regfile ;

ARCHITECTURE behavioral OF regfile IS
TYPE regArray IS ARRAY( 0 TO size-1) OF std_logic_vector (dw-1 DOWNTO 0);
SIGNAL regfile : regArray ;
BEGIN
  PROCESS( clk )
BEGIN
IF ( clk'event AND clk = '0') THEN
IF we = '1' THEN
        regfile ( to_integer ( unsigned( Addr ))) <= A;
END IF?
END IF?
C <= regfile ( to_integer ( unsigned( Addr )));
     END PROCESS?
```

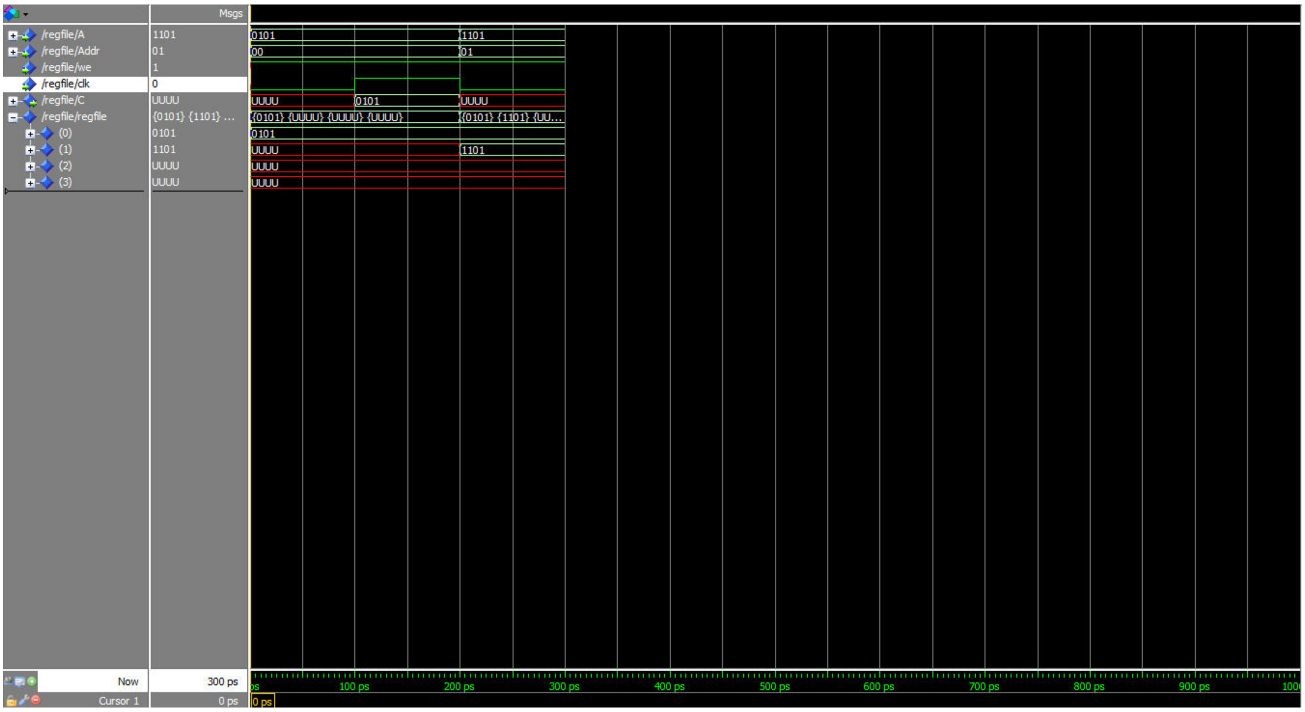
```
END behavioral?
```

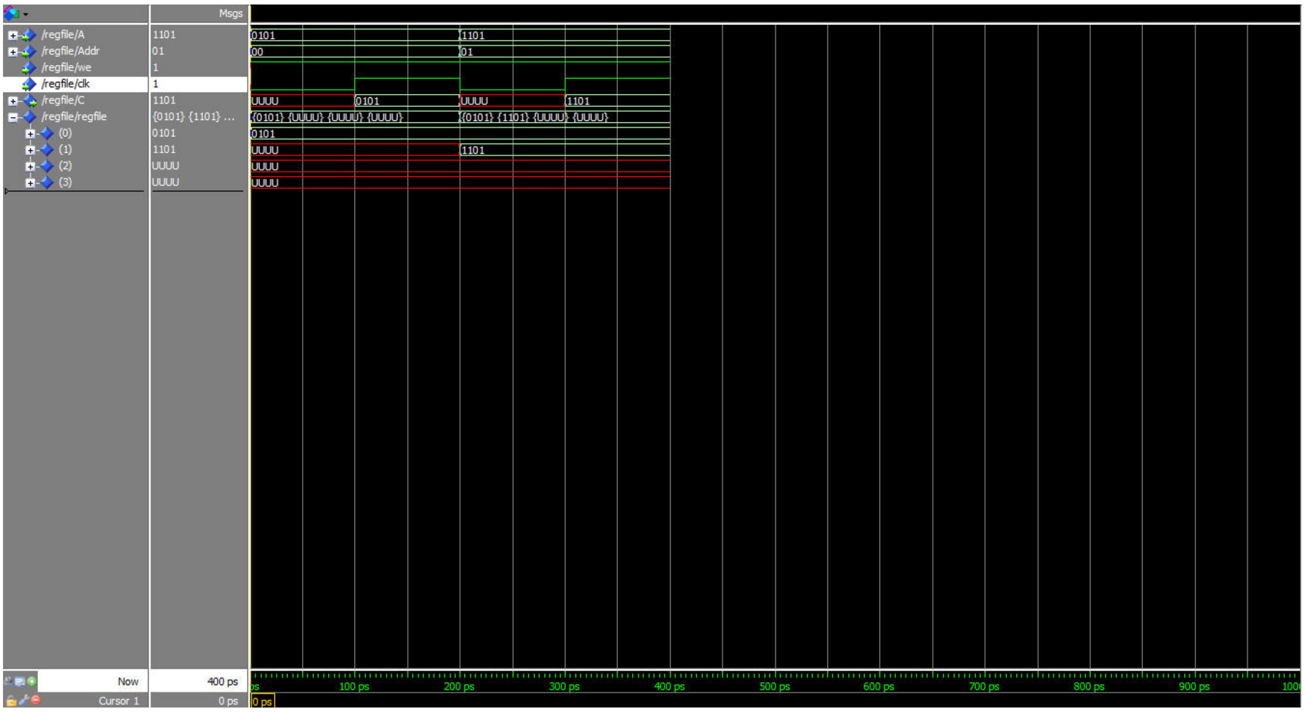
Write 0101 to address 00



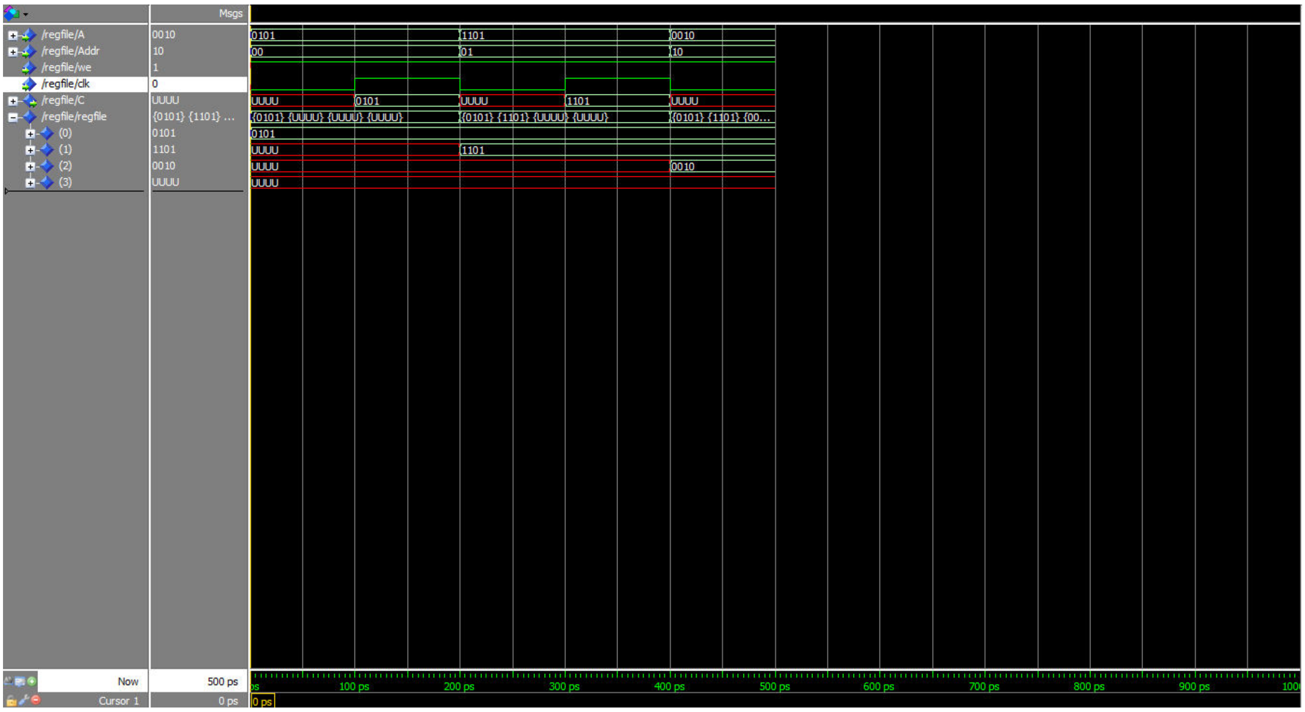


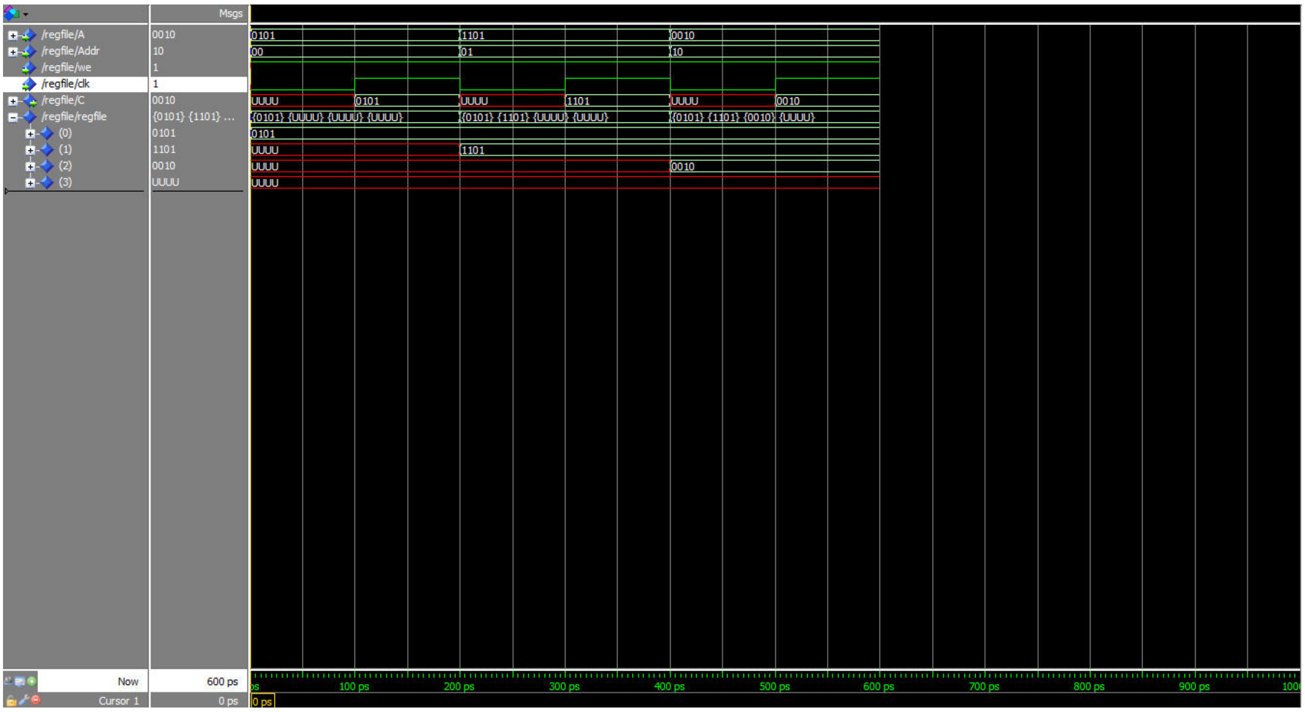
Registration of 1101 at address 01



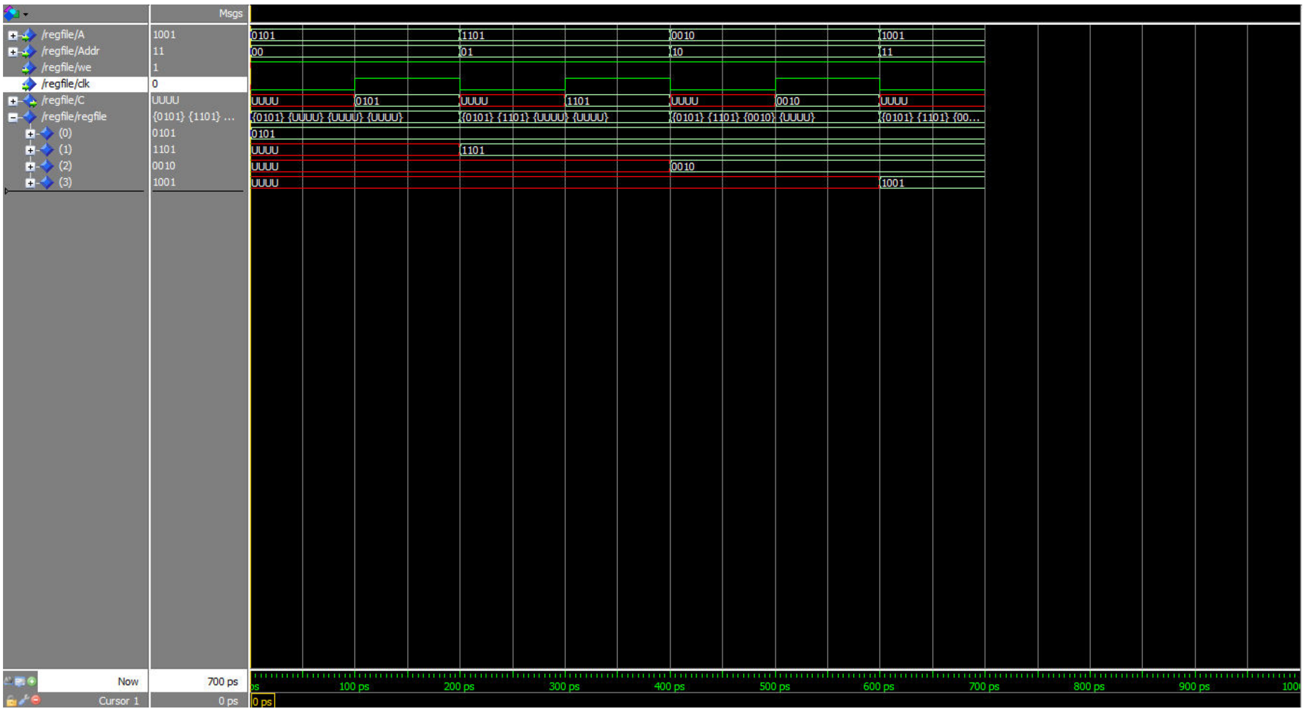


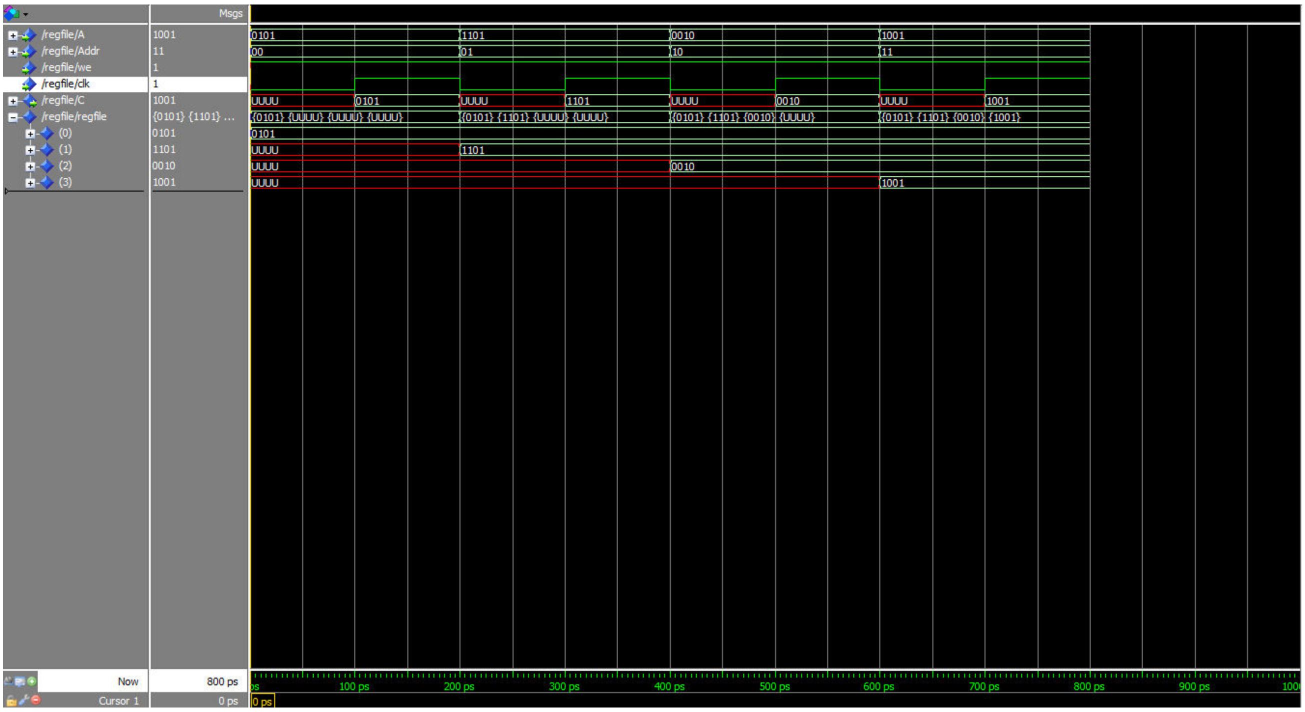
Write 0010 to address 10



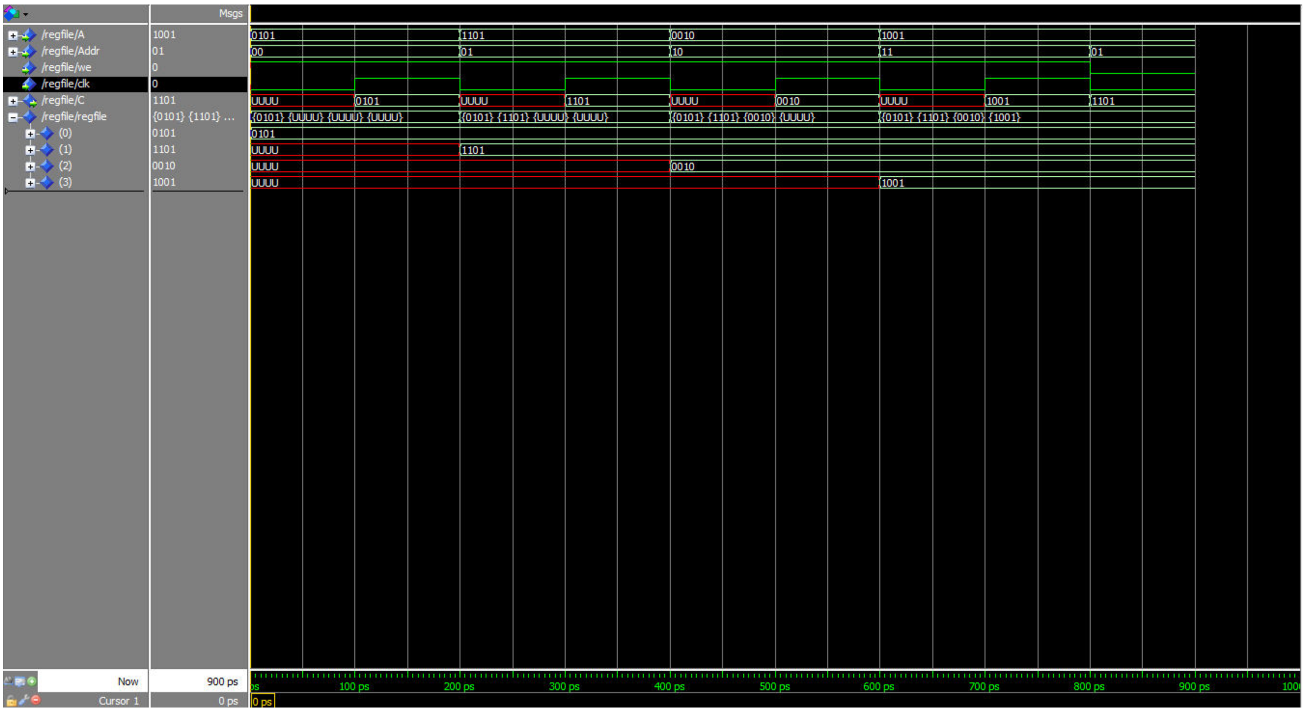


Registration of 1001 at address 11

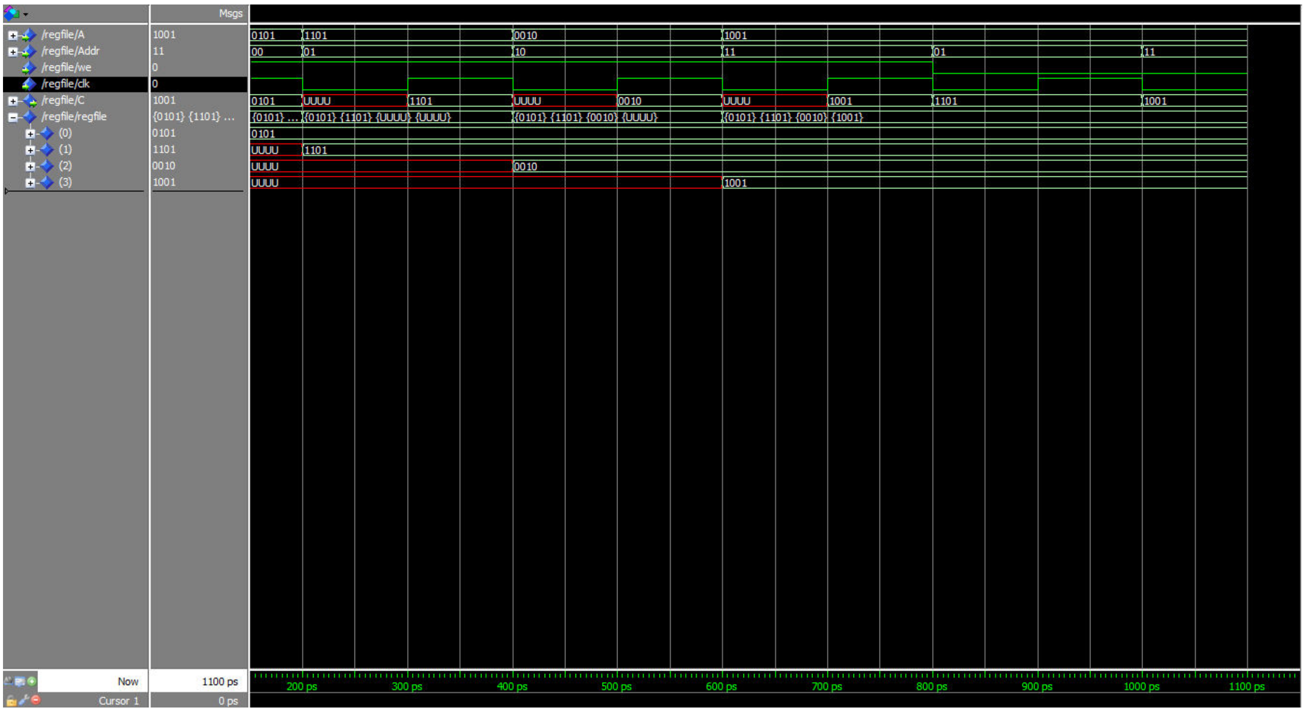




Reading at address 01



Reading at address 11

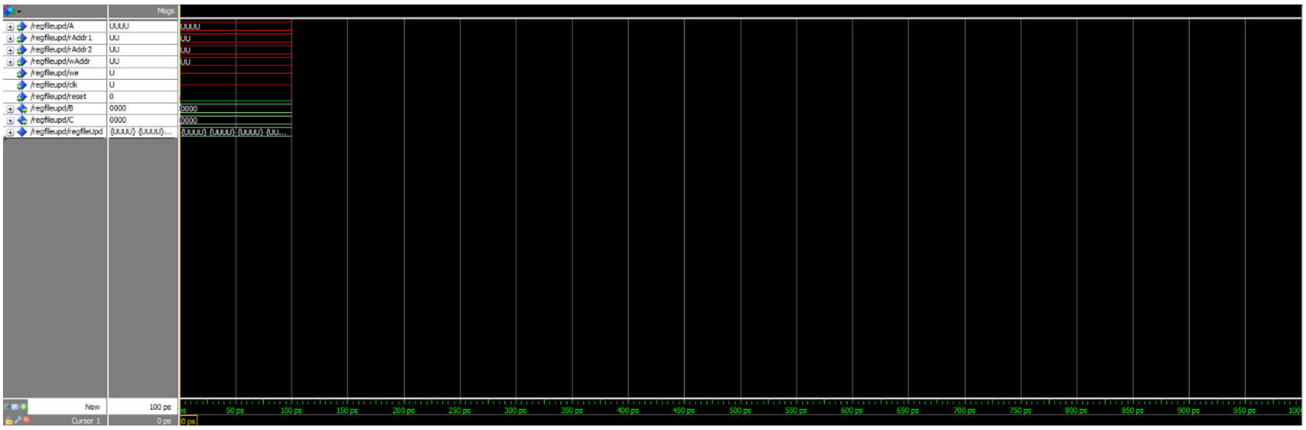


4 . Extend the register file from the previous exercise by adding two additional ports, one read and one write, as well as one additional data input.

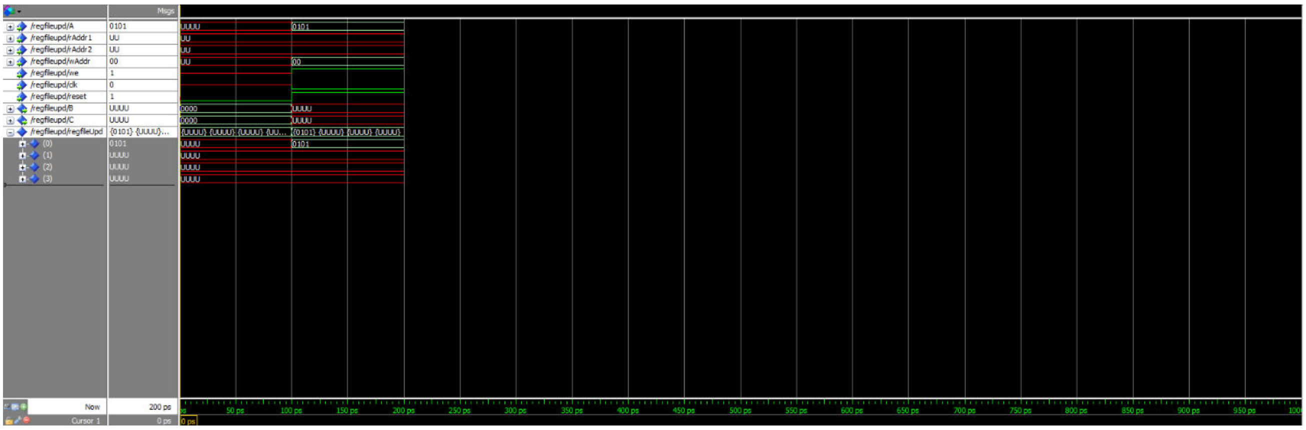
```
LIBRARY ieee ;
USE ieee.std_logic_1164.all;
USE ieee.std_logic_unsigned.all ;
USE ieee.numeric_std.all ;

ENTITY regfileUpd IS
GENERIC ( dw : natural := 4;
            size : natural := 4;
            addrw : natural := 2);

PORT ( A : IN std_logic_vector (dw-1 DOWNTO 0);
rAddr1: IN std_logic_ vector ( adrw-1 DOWNTO 0);
rAddr2: IN std_logic_ vector ( adrw-1 DOWNTO 0);
            wAddr : IN std_logic_vector (adrw-1 DOWNTO 0);
we : IN std_logic ;
            clk : IN std_logic ;
            reset : IN std_logic ;
B : OUT std_logic_vector (dw-1 DOWNTO 0);
C : OUT std_logic_vector (dw-1 DOWNTO 0));
END regfileUpd ;

ARCHITECTURE behavioral OF regfileUpd IS
TYPE regArray IS ARRAY( 0 TO size-1) OF std_logic_vector (dw-1 DOWNTO 0);
SIGNAL regfileUpd : regArray ;
BEGIN
  PROCESS( clk , reset)
BEGIN
IF ( clk'event AND clk = '0') THEN
IF we = '1' THEN
```

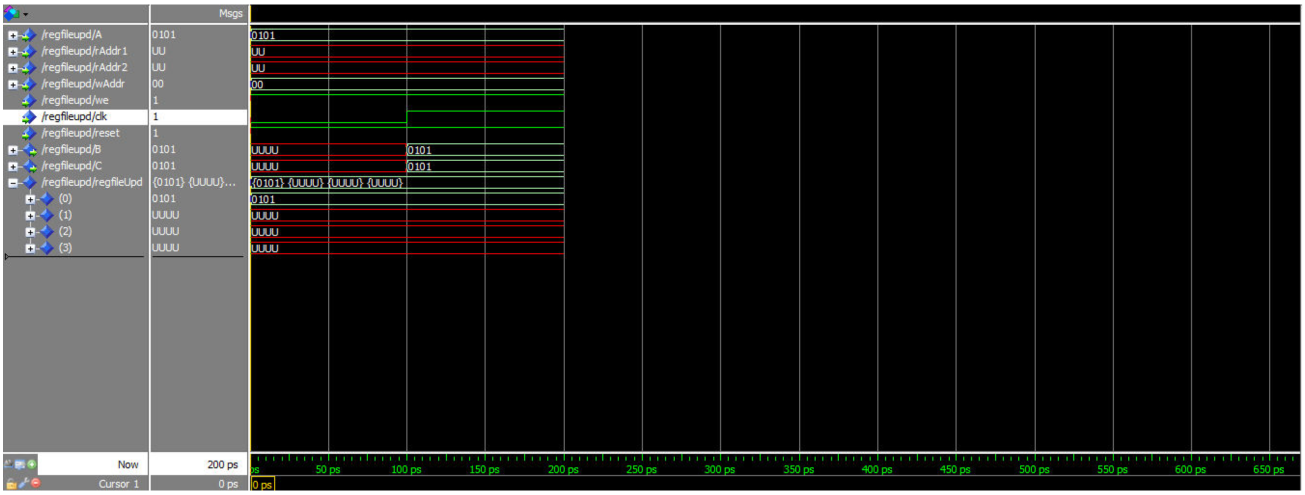
```
        regfileUpd ( to_integer (unsigned( wAddr ))) <= A;
END IF?
END IF?
C <= regfileUpd ( to_integer (unsigned(rAddr1)));
B <= regfileUpd ( to_integer (unsigned(rAddr2)));
IF reset = '0' THEN
C <= "0000";
B <= "0000";
END IF?
     END PROCESS?
END behavioral?
```

Reset

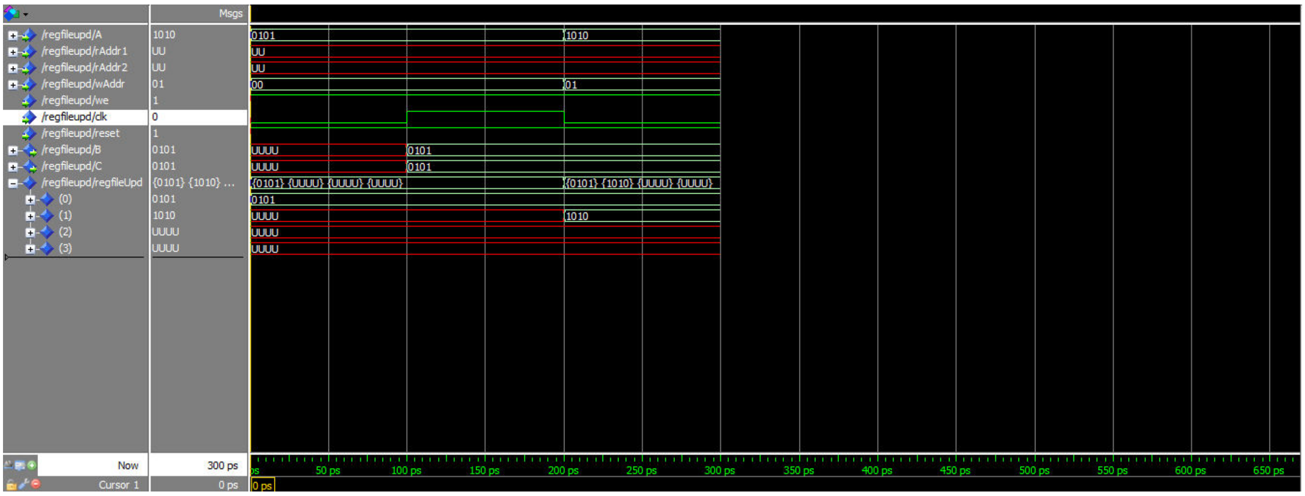


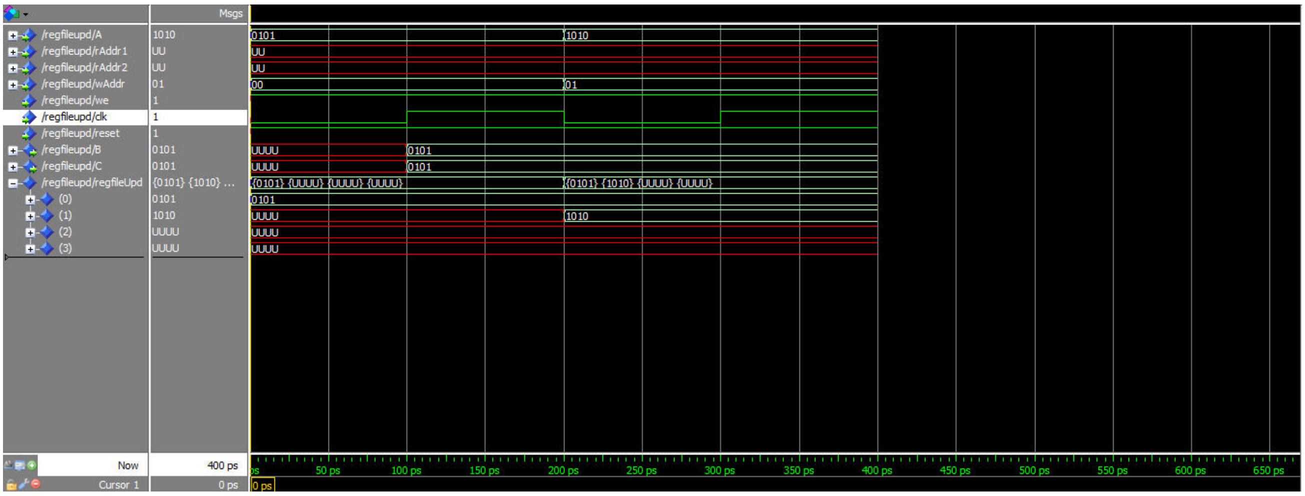
Write 0101 to address 00



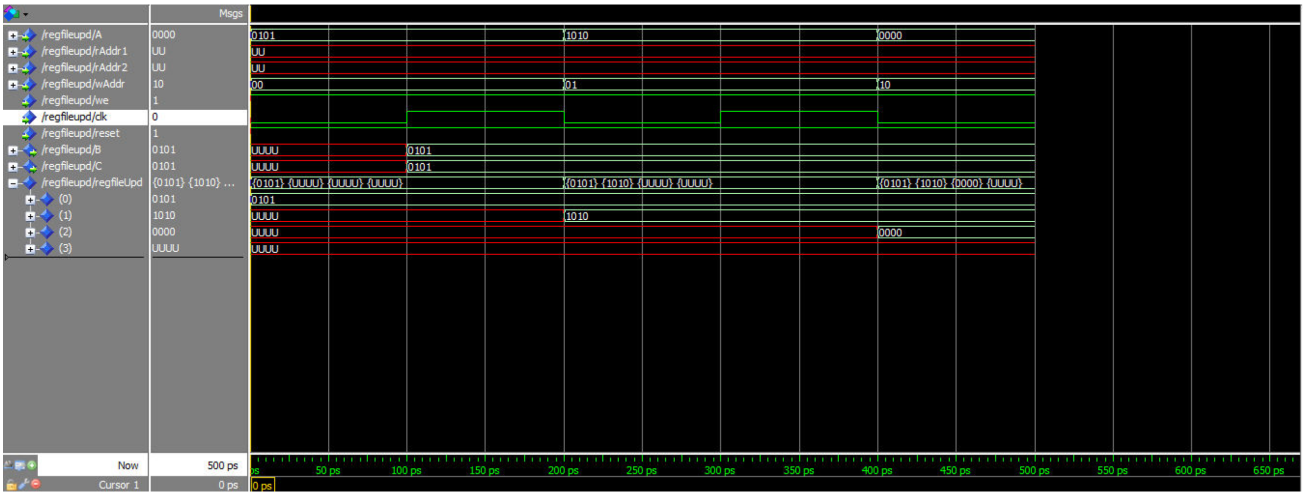


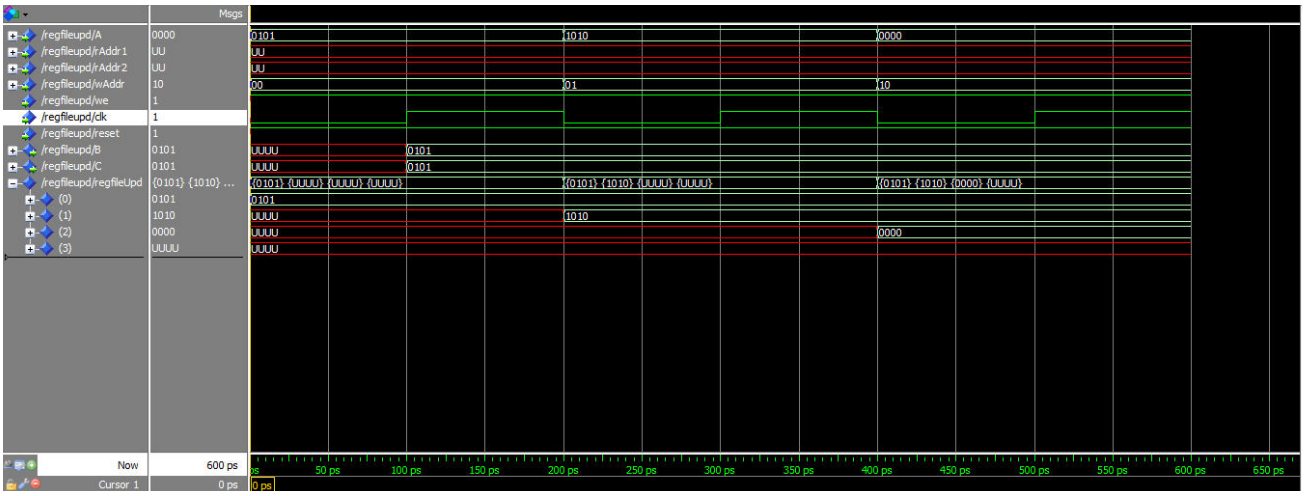
Registration of 101 0 to address 0 1



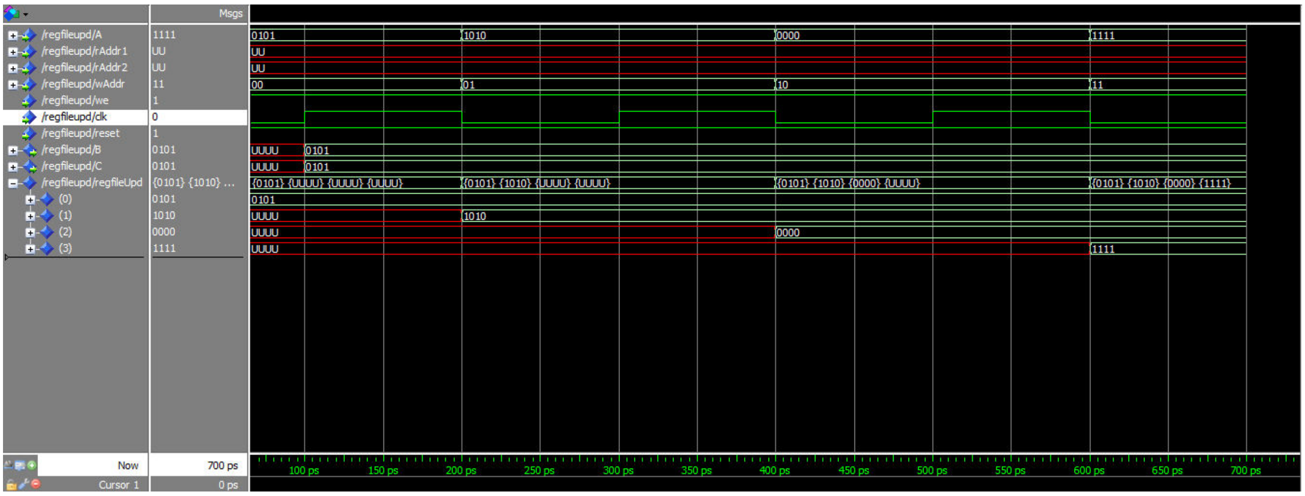


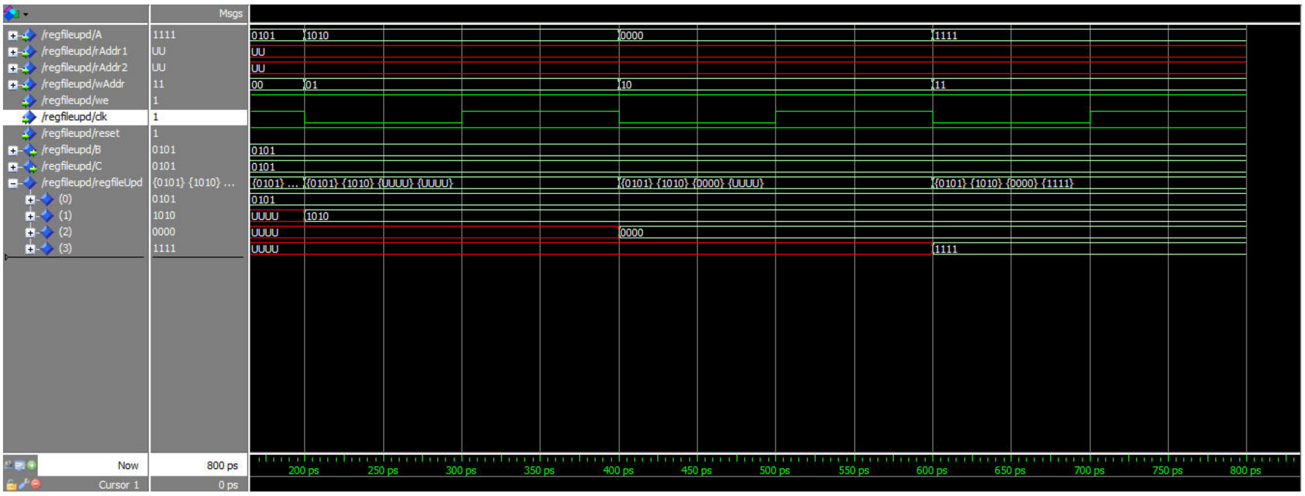
Writing 00 0 0 to address 10



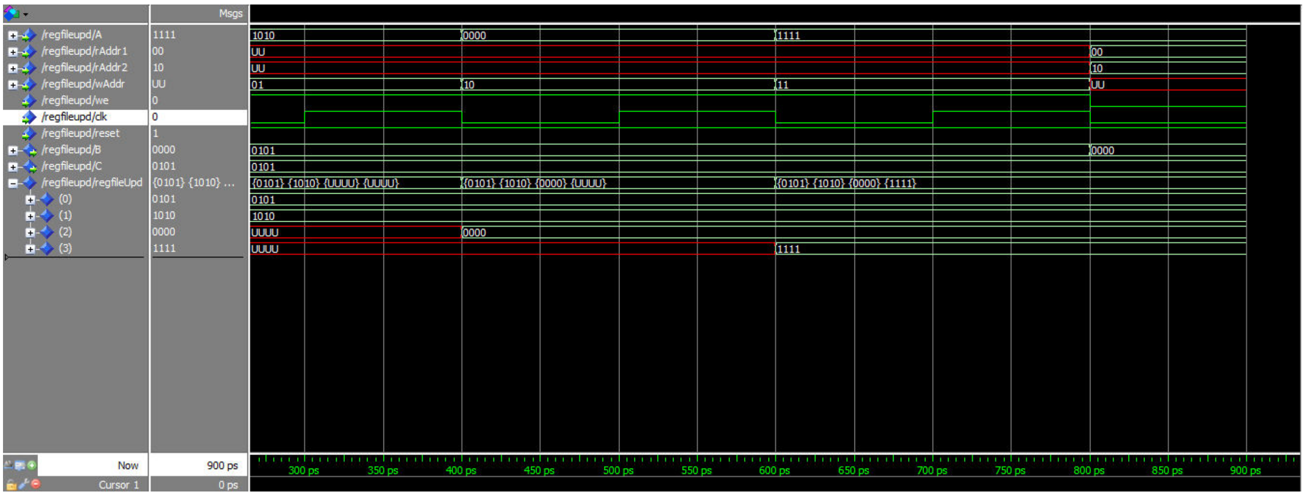


Write 1 1 1 1 to address 11

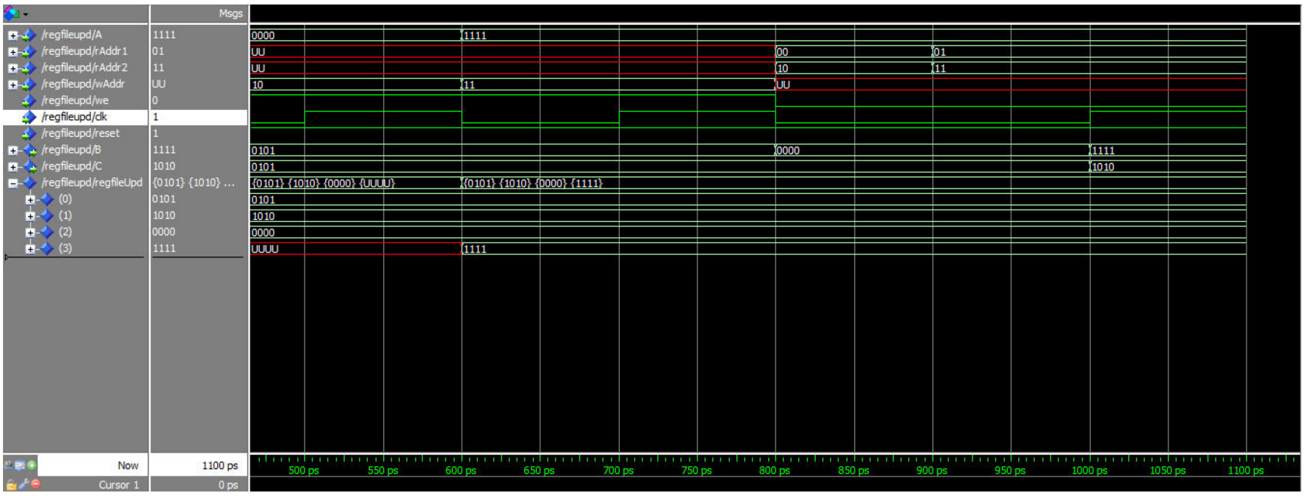




Reading at address 0 0 and 10



Reading at address 01 and 11



Thank you for your attention.